```
const path = require('path');
const { getMovies } = require('./graphql-client.js')
const { Pact, GraphQLInteraction, Matchers }
= require('@pact-foundation/pact');
const { eachLike } = Matchers;
```

We need the path module to manage the file directory naming for our contract because we'll need to store the contract that Pact will generate. We also require the actual query we need to test, which is the `getMovies` function from the graphql-client.js file. Finally, we require the `Pact` and `Matchers` modules from the Pact library, as well as `GraphQLInteraction`, a lightweight wrapper provided by Pact, to make GraphQL contract testing easier.

Unlike the consumer contract testing you first saw in chapter 4, the GraphQL interaction uses PactV2 instead of PactV3. There are breaking changes introduced in PactV3, which you can find at https://mng.bz/Bgq8, but the overall concept of writing the contract test remains the same in PactV2. As of writing this book, there is currently an open request from the community for GraphQL to support PactV3, which you can see at https://mng.bz/NBE1.

7.3.2 Setting up the mock provider

The next step is to define the mock provider that Pact will use, which you should now be familiar with. Add the following lines of code to your graphql-client-contract.spec.js file:

```
const provider = new Pact({
  port: 4000,
  dir: path.resolve(process.cwd(), 'pacts'),
  consumer: 'GraphQLConsumer',
  provider: 'GraphQLProvider',
});

describe('GraphQL example', () => {
  beforeAll(() => provider.setup());

  ...
});
```

This code calls the `Pact` constructor to create a new mock provider that we can use as part of our GraphQL consumer contract test. As part of the constructor, we're passing parameters to define the port for the mock